Transitioning Blocker Calculations: From SPLAT to LoCoS BEP

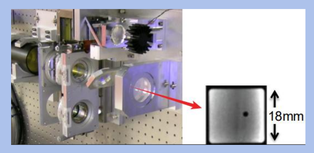
Taylor Bauer[1,2,3], Joyce Li[1,2], Arsenia Rendon[1,2], Cathy Chang[1,2]

[1] Lawrence Livermore National Laboratory - National Ignition Facility, [2] Computing Directorate, [3] Iowa State University, Department of Computer Science

BACKGROUND

*The **Location and Component State Tracking System (LoCoS)** comprises a suite of applications that enable operations to oversee configuration and work activities at the **National Ignition Facility (NIF)**. To achieve ignition, NIF relies on optical components to deliver powerful laser energy. However, the intense laser light from this process causes microscopic damage to the exit surfaces, which must be addressed over time. To mitigate such damage, **Programmable Spatial Shaper (PSS) blockers** are used to modulate the laser within the optical beam path, helping to preserve the integrity of the optics during exposure. As a result, PSS blockers create light shadows on downstream optical elements.*

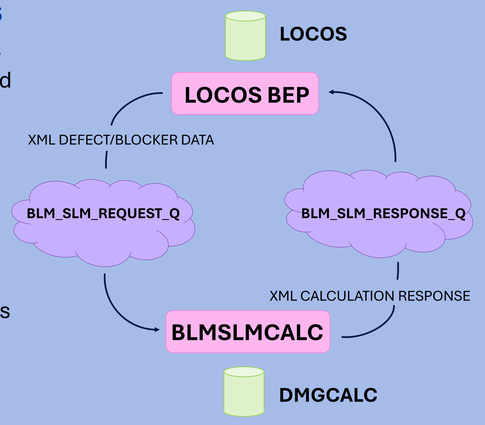
*The **LoCoS Back-End Process (BEP)** is now responsible for optimal blocker placement and calculating the resulting shadows using **the Blocker Location Module** and **Shadow Location Module Calculation (BlmSlmCalc)**. Previously, these calculations were handled externally by the **Shot Planning & Analysis Tool (SPLAT)**. This project aims to port the relevant code from SPLAT into the LoCoS BEP application.*



ASYNCRONUS TO SYNCHRONUS PROCESS

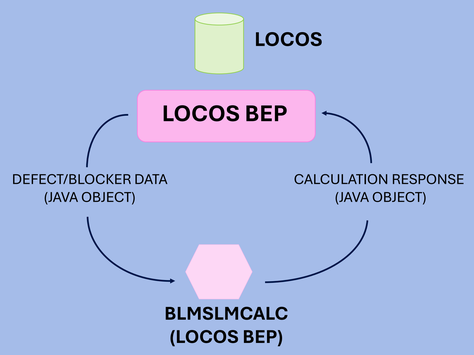
Calculation + Data Exchange Via Queue & JMS

- Optics undergo external inspection protocols, Non-Conformance Reports (NCRs) are created to identify defect parameters
- PssAggregateService aggregates defect metadata (id, coordinates, beamline data) to determine blocker NCR requirements
- A post-shot request is sent to LoCoS to determine if shadow NCRs are needed
- Data is converted to XML and LoCoS generates a calculation request using the Oracle Advanced Queuing (AQ) and Java Messaging Service (JMS)
- BlmSlmCalc dequeues the message and uses DMGCALC schema to compute proper shadow and blocker locations
- The XML calculation response is generated and queued, then processed by LoCoS and stored in its database




```
<blmslmRequestMsg blockerType="PSS" quad="Q11B" source="LOCOS_PSS_AGGREGATOR" test="false">
    <defects>
        <defect avgX="77.39417401716748" avgY="320.27195447722628" id="12765" beam="B11S" opticType="TH0"
                defectNameId="1781920"/>
    </defects>
</blmslmRequestMsg>
```

```
<blmslmResponseMsg blockerType="PSS" errorMsg="" quad="Q11B" source="LOCOS_PSS_AGGREGATOR" test="false">
    <defects>
        <defect id="12765">
            <coordinates>
                <coordinate x="394.60318" y="-129.19556" name="middle_left_fidu_to_block_decimated_pix"/>
                <coordinate x="0.5792" y="0.056" name="beam_center_to_block_rp0_cm"/>
                <coordinate x="394.55" y="-139.29" name="middle_right_fidu_to_block_decimated_pix"/>
                <coordinate x="-141.390" y="-221.546" name="lower_left_fidu_to_block_decimated_pix"/>
            </coordinates>
        </defect>
    </defects>
```



LoCoS BEP Calculation

- Same external inspection and aggregation process
- No longer converted to XML – now uses direct method calls like runBlockers() and runShadows()
- Results stored directly in LoCoS database eliminating messaging and queuing complexity

CODE MIGRATION

Creating Database Objects in LoCoS

- DMGCALC schema to LoCoS schema
  - Tables, Views, PL/SQL packages, and types
- Synonyms to reference remote schemas for data access

Switch to JPA Objects & Spring Data Repositories

- BlmSlmCalc calls Aggregator.java which populates a calculation model
- Replaced JDBC objects with JPA entities used in the Aggregator
  - OIGamma, OpticQness, AASpots – coordinate transformation, alignment, and optic quality data
- JPA repositories replaced prepared statements

```
    PreparedStatement statement = null;
    try {
        statement = dbConnection.prepareStatement("select * from table(BLM_SLM_SELECTOR_PDS.get_gamma_transforms(?,?))");
        statement.setString(1, this.quadName);
        statement.setString(2, this.beamFate.name());
```

```
public interface DBOIGammaRepository extends JpaRepository<DBOIGamma, String> {
    @Query(value = "select * from table(BLM_SLM_SELECTOR_PDS.get_gamma_transforms(:quad, :blockerType))", nativeQuery = true)
    List<DBOIGamma> findByQuadAndBlockerType(@Param("quad") String quad, @Param("blockerType") String blockerType);
```

Testing

- Limited matching data in development environment
- Faux response coordinates to verify dataflow before integrating calculation logic
- Mockito and @MockBean for unit testing without database dependencies

```
when(gammaRepo.findByQuadAndBlockerType( quad: "Q31T",  blockerType: "PSS")).thenReturn(Collections.emptyList());
when(AAspotsRepo.findByQuad("Q31T")).thenReturn(Collections.emptyList());
when(opticQnessRepo.findQnessByOpticType("GDS")).thenReturn(Collections.emptyList());
```

CLASS EVOLUTION

BLMSLMCALC (old)
- initialize()
- run()
- saveBlockers()
- saveShadows()

Before: Single Class Design

- Mixed blocker/shadow logic in run()
- Redundant if/else branching
- Around 500 lines of code
- Poor readability and maintainability

BLMSLMCALC (new)
- runBlockers()
- runShadows()
- **BlockerCalculation**
  - initBlockerCalc()
  - populateModel()
  - calcBlockers()
  - processBlockers()
- **ShadowCalculation**
  - * Shadow calculation has not been migrated yet

After: Modular Design

- Two distinct request sources for calculation
  - Blocker: PSSAggregateService
  - Shadow: Laser Performance Operations Model (LPOM), post shot
- Dedicated services for blocker and shadow calculation

TECH STACK

IMPACT

Code Reduction

- Eliminated over thousands of lines of unnecessary code
- Reduce the queue's XML parsing and mapping classes by over 10 redundant instances
- No more duplicate data conversions (e.g., DefectParameter → Defect for XML)
- Improved data flow clarity and streamlined communication, while also enhancing maintainability

```
public class DefectParameter {

    private static final String BEAM_PREFIX = "B";  2 usages

    private Long id;
    // Defect.defectNameId
    private String defectNameId;

    // Defect.opticType
    private String opticType;

    // Defect.beamline
    private String fbeam;

    private double avgDefectDrawingX;
    private double avgDefectDrawingY;
```

```
public class Defect {
    @JacksonXmlProperty(localName = AVG_X, isAttribute = true)
    private String avgX;
    @JacksonXmlProperty(localName = AVG_Y, isAttribute = true)
    private String avgY;
    @JacksonXmlProperty(localName = ID, isAttribute = true)
    private String id;
    @JacksonXmlProperty(localName = BEAM, isAttribute = true)
    private String beam;
    @JacksonXmlProperty(localName = OPTIC_TYPE, isAttribute = true)
    private String opticType;
    @JacksonXmlProperty(localName = DEFECT_NAME_ID, isAttribute = true)
    private String defectNameId;
```

```
public BlmSlmRequest(BlockerConfigParameter parameter) {  2 usages
    this.quad = parameter.getQuad();
    this.test = parameter.isMessageTest();
    this.blockerType = parameter.getBlockerType();
    this.defects = copyDefects(parameter.getDefectParameters());
    this.source = parameter.getSource();
}

private List<Defect> copyDefects(List<DefectParameter> defectParams) {  1 usage
    if(CollectionUtils.isNotEmpty(defectParams)) {
        return defectParams.stream()  Stream<DefectParameter>
                           .map(this::mapToDefect)  Stream<Defect>
                           .toList();
    }
    return Collections.emptyList();
}

private Defect mapToDefect(DefectParameter param) {  1 usage
    var defect = new Defect();
    defect.setId(Objects.toString(param.getId(), nullDefault: null));
    defect.setDefectNameId(param.getDefectNameId());
    defect.setBeam(param.getFbeam());
    defect.setOpticType(param.getOpticType());
    defect.setAvgX(Objects.toString(param.getAvgDefectDrawingX(), nullDefault: null));
    defect.setAvgY(Objects.toString(param.getAvgDefectDrawingY(), nullDefault: null));
    return defect;
```

```
public Object fromMessage(Message message) throws MessageConversionException {
    try {
        var payload = (String) super.fromMessage(message);
        var messageId = message.getJMSMessageID();
        log.info("Convert from message: {}", messageId);
        var correlationId = message.getJMSCorrelationID();
        log.info("CorrelationId: {}" , correlationId);
        log.debug(payload);
        var blmResponse = readXml(payload, BlmSlmResponse.class);
        var blockerResponse = blmResponse.addBlockerResponse();
        blockerResponse.setMessageId(messageId);
        blockerResponse.setCorrelationId(correlationId);
        return blockerResponse;
    } catch (JMSException e) {
        throw new MessageConversionException(e.getMessage(), e);
    }
}

@Override  3 usages
public Message toMessage(Object object, Session session) throws MessageConversionException {
    try {
        var blmRequest = new BlmSlmRequest((BlockerConfigParameter) object);
        var mdl = toBdl(blmRequest);
        log.debug(mdl);
        return super.toMessage(mdl, session);
```

New Features & Testing

- With improved modularity, methods and components can be tested separately, and integrating shadow calculation or future enhancements is more straightforward

FUTURE IMPLEMENTATIONS

Extend BlmSlmCalc to handle SLM calculation

- Occurs post-shot
- Migrate database objects & calculation logic
- Same direct method call – runShadows()

Email Service

- Notify users if an error has occurred during calculations

Frontend/GUI additions

- BlockerAggregator GUI shows status (Pending, Calculated, etc.) along with other info
- Potentially add more features